
REAL-TIME COLLISION RISK PREDICTION ON EDGE DEVICES: OPTIMIZATION AND VALIDATION FOR EDGE CASE SCENARIOS

Final Report of the Graduation Project

Jawahar Pranav, Udhayakumar
1867202

Supervisor
Dr. Gijs Dubbelman

Eindhoven, November 2024



Automotive Technology
Signal Processing Systems

Real-Time Collision Risk Prediction on Edge Devices: Optimization and Validation for Edge Case Scenarios

Jawahar Pranav, Udhayakumar

j.p.udhayakumar@student.tue.nl

Abstract—This study presents an optimized real-time collision risk prediction model for deployment on edge devices in autonomous vehicles. Building upon the RiskNET framework, we implement hardware-specific optimizations using NVIDIA TensorRT to achieve low-latency inference on an NVIDIA Jetson AGX Orin platform. Our approach integrates YOLOv10 for efficient object detection and leverages GPU memory optimizations to reduce inference time. We validate the model's performance through a Hardware-in-the-Loop (HIL) setup using Siemens' Simcenter Prescan, testing across diverse simulated scenarios including highway, urban, suburban, and rural environments under various weather conditions. Results demonstrate consistent performance with average precision scores ranging from 0.919 to 0.994 across scenarios. The study reveals that while significant improvements in collision avoidance are achievable, the model's one-second prediction window remains a limitation for anticipating longer-term risks. Our findings highlight the potential of edge-optimized models for real-time collision risk prediction, while also identifying areas for future improvement, including extending the prediction timeframe and enhancing model robustness in complex traffic scenarios.

I. INTRODUCTION

TRAFFIC accidents continue to pose a significant threat to public safety, resulting in considerable loss of life and property damage each year. As vehicle numbers increase, the need for effective safety measures has become more urgent. Advanced technologies, particularly those found in driver assistance systems, are crucial in preventing collisions. These systems leverage artificial intelligence and real-time data to anticipate potential hazards and intervene when necessary. However, challenges remain in ensuring these technologies perform reliably in complex traffic scenarios. As the automotive industry moves toward greater automation, the development of sophisticated collision avoidance systems that can adapt to dynamic driving conditions is essential for enhancing road safety [1]. Autonomous Emergency Braking Systems (AEBS) represent one of the most critical advancements in collision avoidance technology. These systems are designed to automatically detect potential frontal collisions, warn the driver, and apply emergency braking if the driver fails to respond in time [2]. The significance of AEBS is reflected in various regulatory frameworks, with organizations like the European New Car Assessment Programme (Euro NCAP) making it a key criterion in their safety ratings [3]. Traditional AEBS

implementations rely on a combination of sensors including radar, LiDAR, and cameras, processing their inputs through deterministic algorithms with pre-defined thresholds for risk assessment and brake actuation [4]. In this research, we work on enhancing this task by picking our baseline from the collision risk prediction model ("*RiskNET*") proposed by Schoonbeek *et al.* [5][6].

Current industry standards for collision avoidance systems heavily depend on Radars for depth measurements coupled with RGB cameras to classify traffic agents [7][8][9]. These collision avoidance systems are designed to warn drivers of high collision risks and can take over driving tasks, such as applying emergency brakes when the risk surpasses a certain threshold. Even though these systems promise to prevent most rear-end collisions, their effectiveness is limited to simple movement patterns, such as when the vehicle is approaching slow or stationary traffic on highways. The focus of some research is on addressing the challenges posed by highly dynamic and complex movement patterns typical of rural and urban roads. While accurate model-based approaches are often too complex for real-time application, traditional real-time methods lack precision due to necessary simplifications. Advanced data-driven approaches, particularly end-to-end systems using camera images to predict collision risks, represent a pinnacle in this field [10].

Our baseline RiskNET is a neural network model that implements an end-to-end pipeline that processes RGB images from a pre-saved and pre-processed dataset to generate an intermediate attention mask representation. This attention mask incorporates generalized depth information of surrounding road users derived from their respective pre-saved bounding box coordinates predicted by an object detection model. RiskNET is trained and evaluated using these attention masks. Ultimately, the pipeline predicts a safe or an unsafe scenario according to the situation projected by the input RGB images. However, this setup is not suitable for real-time collision risk prediction, as it relies on pre-saved bounding boxes. For real-time applications, all pre-processing steps, including bounding box prediction and attention mask generation, must be executed instantaneously, with as low a delay as possible. These technically render our implementation of RiskNET into twofold: the first component involves real-time pre-processing tasks such as object detection and attention mask generation, while the second component focuses on

This work was partly supported by VDL Enabling Transport Solutions, Helmond & Siemens Industry Software, Helmond, under the "*Digitale infrastructuur voor Toekomstbestendige Mobiliteit*" (DITM) project.

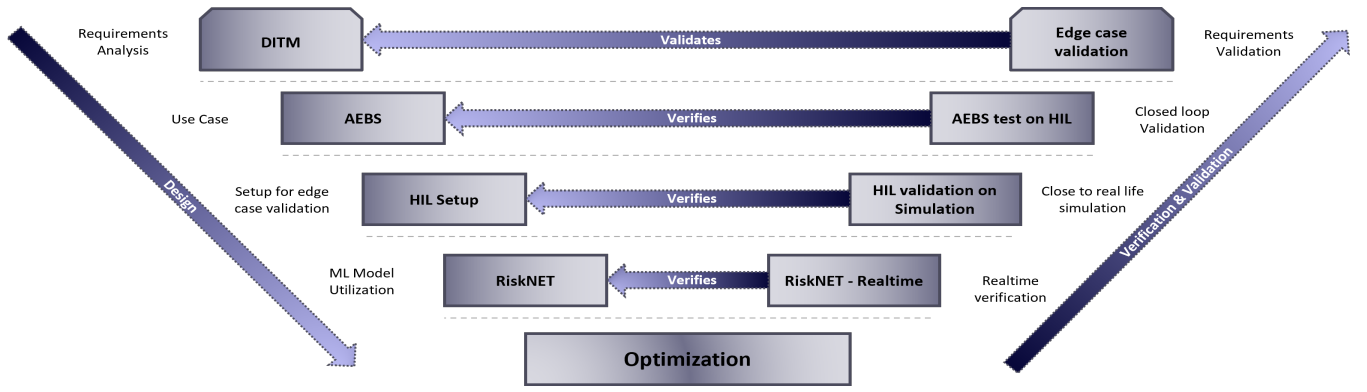


Fig. 1: V-model based research approach

real-time collision risk prediction.

In the past decade, deep learning has made remarkable strides and has been applied in various fields, including speech recognition, natural language processing, and computer vision. Computer vision technology has been widely adopted in domains like general perception systems, autonomous driving, intelligent transportation, and defense monitoring [11]. Within computer vision, tasks are typically classified into image classification, object detection, and image segmentation. Notably, object detection, has experienced significant advancements, largely due to the availability of extensive object detection datasets such as MS COCO [12], PASCAL VOC, and Visdrone. The knowledge from these findings are used to pick the best model for our pipeline’s object detector module, with appropriate optimization.

ImageNet’s classification leader board provided by the papers with code [13] reports, state-of-the-art models for image classification have expanded from 61 million parameters to 2100 million since 2013. While these large models achieve high accuracy and precision, they often suffer from slow inference times and require significant computational resources, posing challenges for real-time applications and deployment on resource-constrained devices like embedded and edge systems [14]. This limitation is particularly significant in applications such as real-time collision risk prediction, where split-second decisions are essential. For example, the design standards set by the American Association of State Highway and Transportation Officials (AASHTO) allocate only 1.5 seconds for perception time when calculating stopping sight distance [15]. This narrow window underscores the necessity for rapid processing and decision-making in automotive safety systems, highlighting the importance of efficient, low-latency models capable of functioning within these stringent time constraints.

Within the scope of this research, the objective is to carry out real-time collision risk prediction using edge devices. This necessitates processing the frames captured by the vehicle’s cameras as fast as possible to prevent collisions with obstacles, underscoring the importance of deploying DL models on embedded and edge devices capable of real-

time computation within operational environments. Model optimization is imperative in addressing these challenges, emphasizing the need to consider hardware constraints and environmental sustainability [16]. Techniques such as pruning [17], quantization [18], and knowledge distillation reduce model size and computational requirements, enabling efficient deployment on devices with limited resources. These optimizations not only enhance computational efficiency but also decrease energy consumption and carbon footprint [19]. However, meticulous optimization tailored to specific hardware capabilities is critical. Without it, models may under perform or fail on new platforms, leading to inefficient resource usage and increased usage costs.

Our thesis is structured according to the V-model Fig.1, which demonstrates a systematic approach to this research. The left side of the V, covering the design of the project requirements (DITM) and AEBS, is addressed in Sections 1 and 2, where foundational concepts and system configurations are established. Section 3, located at the bottom of the V, outlines the methodology through the HIL setup and the implementation of RiskNET, with a focus on optimizing real-time performance. The right side of the V, explored in Sections 4, 5 and 6, involves the verification and validation of the system through HIL simulations and edge case testing, ultimately confirming that RiskNET fulfills the design objectives and completes the V-model framework.

II. RELATED WORK

In this section, we discuss our baseline model and the relevant research related to different parts of our proposed real-time pipeline.

A. RiskNET

Our baseline RiskNET [5] proposes an image-based collision risk prediction pipeline, which consists of a perception and a collision risk prediction module. The perception module pre-processes the input modalities from the saved metadata of the dataset, such as the detected bounding box coordinates, masks, etc. The output from this module is ready to be fed into the collision risk prediction model.

The collision risk prediction model is a spatio-temporal 3D CNN that takes in pre-processed attention masks (and other modalities if required) as inputs. These input modalities pass through a single 3D convolution layer with 8 output channels (proportional to the dimension of time), kernel size 3, zero padding, and a stride of 1 to preserve the original input dimensions. The captured features are fed into a 3D ResNext feature extractor with 18 layers inspired by K. Hara *et al.* [20]. ResNexts[21] are basically modified ResNets [22]. In the last step, a fully connected layer acts as a binary classifier and gives out a 0 or a 1, depending on the input scenario.

RiskNET tries to solve the problem of domain gap generalization. Due to the scarcity of real-world datasets, deep neural networks like RiskNET do not have the luxury to train, validate, and infer on real-world datasets. This is where RiskNET's training strategy stands apart. It is trained on simulated video data and is tuned to successfully generalize to real-world data while inferring. It addresses the scarcity of real collision data by using domain-resilient intermediate representations like vehicle bounding boxes and optical flow estimates and separates the learning process from the specific domain. The use of optical flow estimation is limited to simple scenarios where collisions with stationary objects within a short distance are predicted. However, optical flow becomes noisier with distance, making it unsuitable for automotive applications where risks exist at greater distances. Research by Schoonbeek *et al.* [5] shows that RiskNET can effectively learn from vehicle detection sizes and positions over time using attention masks. This makes RiskNET adaptable to different situations, and thus we choose to use vehicle detections as the sole input modality for our modified baseline.

B. Challenge of pre-processing in real time

As proposed by the twofold implementation that was discussed above, the first fold consists of real-time pre-processing of RGB frames from the camera and the generation of intermediary representations. For the required RiskNET input, an object detection model has to detect vehicles in the scene and generate attention masks that are to be fed to the second fold of the pipeline. As per Zhang *et al.* [23], "Deep learning-based object detection algorithms can be broadly classified into two categories. The first category consists of two-stage object detection algorithms that rely on candidate regions for generating potential regions and performing classification and position regression to achieve high-precision object detection. Some of the most popular algorithms in this category include R-CNN, Faster R-CNN [24], Mask R-CNN, and Sparse R-CNN. While these algorithms achieve high accuracy, their slower speed prevents real-time detection on all kinds of devices.

The second category comprises single-stage object detection networks based on regression. These algorithms directly predict the position and class of objects from input images using a single network, avoiding the complex process of generating candidate regions and achieving faster detection

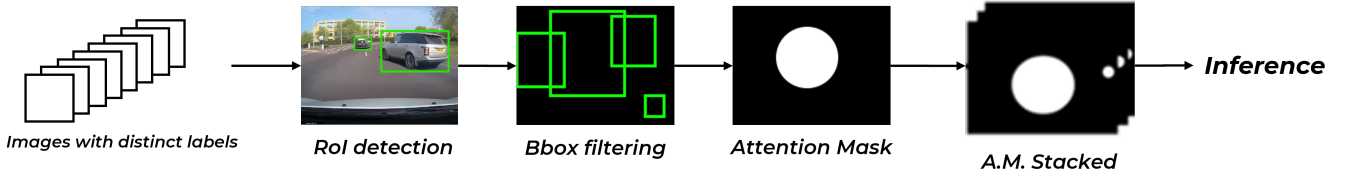
speeds. The main representative networks in this category include SSD and the YOLO [25] series". Even though models like YOLOv5 [23] and YOLOv7 [26] are lighter and faster on edge devices, they do not do Non-Maximal Suppression (NMS) natively. Thus, even when these models are optimized for edge devices, an NMS algorithm has to run alongside the detector to choose the best bounding box for a given detected object. This has a significant impact on the inference latency and thus is not suitable for real-time detections on edge devices. In this research, we explore the latest SOTA YOLOv10 model, as per [27] "it tackles the problem of redundant predictions in post-processing by presenting a consistent dual assignments strategy for NMS-free YOLOs with dual label assignments and consistent matching metrics".

C. Optimization for real-time inference

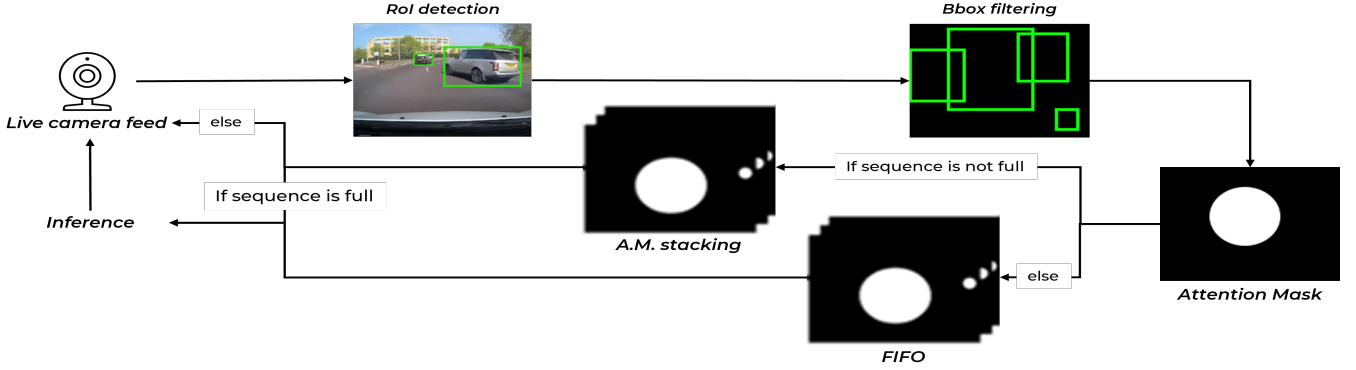
To bridge the gap between the capabilities of large deep learning (DL) models and their practical deployment, a variety of specialized hardware aka Neural Processing Units (NPU) and strategies have been developed. Examples of NPUs include the Apple Neural Engine, Nvidia's Jetson AI edge devices [28], Google's Edge TPU [29] etc. These NPUs focus on accelerating operations such as vanilla convolution, depth-wise convolution, and MLP operations, making them suitable for real-time applications like object detection. These specialized hardware solutions play a crucial role in meeting these requirements by providing the necessary computational power while maintaining efficiency. Moreover, these specialized hardware solutions utilize dedicated entities like tensor and CUDA cores on GPUs to boost model performance. High-performance inference engines like NVIDIA's TensorRT [30] further enhance models trained on major frameworks like PyTorch [31] and TensorFlow, facilitating their deployment on various platforms, including automotive systems. These technologies enable the deployment of DL models on devices with limited resources, ensuring efficient and reliable real-time processing. Specifically, TensorRT performs six types of optimizations to reduce latency and increase the throughput of the model: 1. Weight and activation precision calibration; 2. Layer and tensor fusion; 3. Kernel auto-tuning; 4. Dynamic tensor memory; 5. Multi-stream execution; 6. Time fusion. Zhou *et al.* [14] compares the latency and other performance metrics of a variety of models via different workflows, namely default Python inference and different variations of TensorRT inference packages. Their work clearly outlines the superiority of TensorRT optimized models over default pytorch models. Shin *et al.* [32] introduces a performance inference method that utilizes the Jetson monitoring systems to efficiently run deep learning-related object detection models on Nvidia Jetson platforms.

III. IMPLEMENTING RISKNET - REALTIME

Any baseline model, for continuous real-time inferencing, has to undergo various steps of changes ranging from off-setting overheads to hardware specific optimizations. This



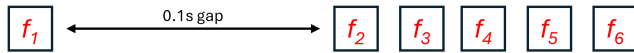
(a) RiskNET (Schoonbeek *et al.* [5])



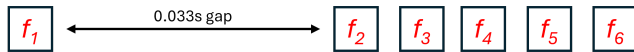
(b) RiskNET - Realtime inferencing pipeline

Fig. 2: Schematic comparison of RiskNET & RiskNET - Realtime: RoI (Region of Interest), Bbox (Bounding Box), A.M. (Attention Mask), FIFO (First In First Out)

research of ours demonstrates the viability of lag-free deployment of RiskNET for real-time collision risk prediction. In the context of our baseline RiskNET, it trains and infers on large datasets in batches. But for real-time inference, the concept a of dataset seizes to exist. The flow of input data instances to the model is in real-time and is generated in proportion to the output frequency of the camera sensor, represented by its frames per second (fps).



(a) 10 fps feed: New frames are generated every 0.1s



(b) 30 fps feed: New frames are generated every 0.033s

Fig. 3: The time gap between two consecutive frames. f_i represents the i^{th} frame in an image feed.

As proposed by Schoonbeek *et al.* [5], the risk prediction model looks at a sequence of images rather than just one image, this way the model takes the spatiotemporal information along with the scene semantics into account. In this case, the model looks at 0.8s worth of footage at once to make one prediction. Thus, originally the datasets were pre-processed to form individual input sequences of $T=8$ images for a 10 fps feed, $T=16$ images for a 20 fps feed, and so on, where T points to the size of the sequence. In order to obtain high-frequency real-time predictions for every frame that the camera captures, i.e.

Number of predictions per second = Number of fps

the batching of input sequences are overlapped. Unlike the original implementation, which infers on a saved dataset, real-time inference does not have the frames from the future. Thus for real-time inference, our new implementation gives out one prediction for every new frame captured by the camera. In that case, it is important to take the overall processing time of the entire pipeline into account. For example, the time taken for the entire pipeline, i.e. frame capturing, bounding box detection of road users, attention mask generation, updating spatiotemporal sequence and finally making a prediction, has to be performed before the next frame arrives for lag-free processing. In the case of inference on a 10 fps feed, this total processing time has to be $\leq 0.1s$ (1 sec $\div$ 10 frames) as shown in Fig. 3a and for a 30 fps feed, the total processing time has to be $\leq 0.033s$ (1 sec $\div$ 30 frames) as shown in Fig. 3b.

A. Architectural modifications

RiskNET originally devised vehicular collision risk prediction as a binary classification problem with a sequence of input RGB images f_i . The predictions are binary (y_i) and gives out a binary classification probability, p_i^{safe} and p_i^{unsafe} . The pipeline can be theoretically represented as,

$$y_i = RiskNet(Perception(f_i)) = (p_i^{safe}, p_i^{unsafe}) \quad (1)$$

where $p_i^{safe}, p_i^{unsafe} \in [0,1]$. Equation 1 defines how Schoonbeek *et al.* defines their prediction pipeline. The perception module is originally defined as the pre-saved

pre-processing unit, which leverages appropriate pre-trained models to estimate the intermediary representations to generate attention masks for their corresponding input frames f_i .

In our implementation for real-time predictions, which we call RiskNET - Realtime, we propose a closed, model in the loop architecture. As shown in Fig. 2b, our pipeline starts by capturing frames from a live camera feed, which samples at a constant fps. As soon as a frame is captured, the regions of interest are estimated by running an object detection algorithm. These RoIs are further filtered out to drop unwanted detections. From these filtered detections, the pipeline proceeds to generate attention masks. For each detected road user, a circular mask is drawn inside its bounding box. It is important to note that attention masks are preferred to be generated from their corresponding object detection bounding boxes rather than from other detection techniques such as semantic segmentation. For real-time implementation, object detections are the fastest way to identify road users while semantic segmentation models can be slower and computationally expensive. Once these attention masks are generated, they are appended to the spatiotemporal sequence.

When this pipeline is initially activated, the sequence will lack the necessary number of attention masks to meet the spatiotemporal requirement, T . In this case, the attention masks are appended to an empty sequence. The above procedure repeats until the sequence is filled with T number of attention masks. Once it is full, this sequence is sent to make one live prediction. At this stage, the sequence has T number of attention masks stacked together with the T^{th} item being the attention mask generated from the latest frame captured by the camera sensor. This also means that our new implementation will start making predictions from the T^{th} frame onwards (f_T). When the camera sensor captures the $T + 1^{th}$ frame, the sequence is already full. We implement the First In First Out (FIFO) mechanism to update the sequence with the latest frame f_i captured. Thus, every time a new attention mask is generated, the attention mask in the 0^{th} index is deleted and the latest attention mask is appended to the T^{th} index. And once again the sequence is full and is sent to make one live prediction. The sequence at all times can be theoretically represented as,

$$S_i = \{A_{i-T}, \dots, A_i\}$$

where S_i and A_i are the sequence and attention mask at instance i . In the context of real-time inference, maximizing performance is crucial. Therefore, to fully utilize computational memory for model initialization and processing, no intermediary metadata is preserved. For instance, frames and bounding boxes are not retained, as they are used exclusively for generating attention masks before being promptly removed from memory. Consequently, the only retained data is the attention mask.

B. RoI detection

For our real-time object detection tasks, we selected YOLOv10-S (Small), which offers an optimal balance between accuracy and inference speed. YOLOv10 introduces several key advancements over its predecessors:

- **NMS-Free Detection:** Utilizes consistent dual label assignments, eliminating the need for non-maximum suppression during inference, thereby reducing latency.
- **Enhanced Efficiency:** Employs lightweight classification heads and spatial-channel decoupled downsampling, significantly reducing computational requirements.
- **Improved Accuracy:** Incorporates partial self-attention modules and large-kernel convolutions, enhancing feature capture without compromising speed.

YOLOv10-S, in particular, provides the right trade-off between detection accuracy and inference speed for our real-time applications. Its compact architecture allows for efficient deployment on resource-constrained systems while maintaining competitive accuracy on standard benchmarks.

C. TensorRT Optimization

For optimized inference, we accelerate both, our object detection model and RiskNET using TensorRT. NVIDIA TensorRT is a comprehensive suite of APIs designed for efficient deep learning inference. It features an inference runtime and various model optimization techniques that ensure low latency and high throughput, making it ideal for real-time inference. To optimize our models, we pick their best pre-trained checkpoints. For the object detection model, we use the COCO [12] pre-trained YOLOv10s checkpoint [27] and for our risk prediction model, as ablated by Schoonbeek *et al.*, we use the checkpoint with the highest AP on the Youtube Driver's Incident dataset.

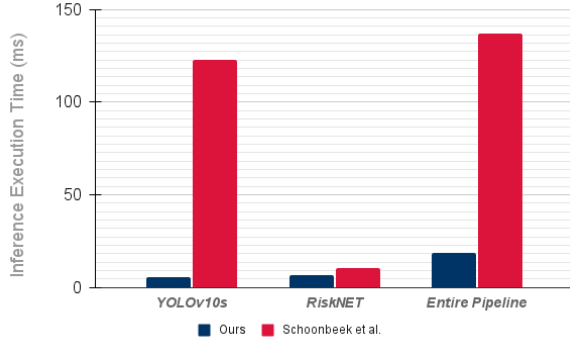
To infer in a TensorRT accelerated pre-trained model, there are three workflows to integrate TensorRT [14]. They are 1) Torch-TensorRT 2) ONNX to TensorRT conversion and 3) ONNX Runtime with TensorRT Integrated. Results published by Zhou *et al.* particularly for ResNet-based models, show that ONNX to TensorRT conversion has the best overall performance for improving PyTorch model inference and hence we choose to integrate TensorRT using this workflow.

We can highlight the key components of this workflow in three discrete steps. It starts with exporting the PyTorch model to an ONNX file, then building this ONNX file into a TensorRT engine and finally deploying this TensorRT engine for inference. To start with, we export the PyTorch model to ONNX files using the `torch.onnx.export()` function from the PyTorch library. This function takes an input tensor to trace the model's execution and then exports the traced model to an ONNX file. By default, the input size is fixed in the exported ONNX graph for all input dimensions. To export an ONNX model that can accept input tensors of dynamic batch size, we need to specify the first dimension as dynamic in the dynamic axes parameter of the `torch.onnx.export()`

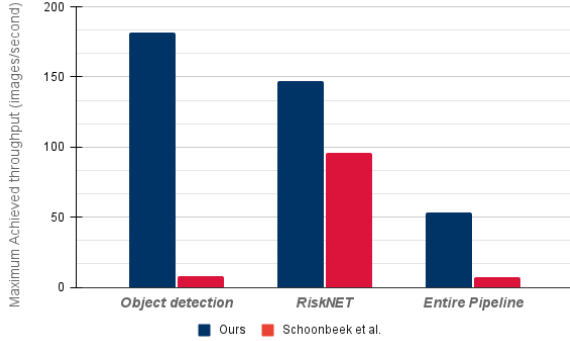
function. We export PyTorch models to ONNX files with a static input shape, as our model pipeline converts all input camera feeds to a fixed downscaled resolution for simple and fast processing. We construct the TensorRT engine with an ONNX model using the TensorRT Python API (trtexec). The construction stages involve operations such as creating a network definition, importing a model through the ONNX parser, and building a TensorRT engine with a builder [33].

D. Hardware Platform

To deploy our model for real-time inference, our embedded computing device of choice is the NVIDIA Jetson AGX Orin 64GB Developer Kit for our experiments. It features a state-of-the-art architecture including 2048 CUDA cores and 64 Tensor Cores GPU, alongside a 12-core ARM processor, peaking at 275 TOPS [34].



(a) Mean inference execution time for all networks (↓)



(b) Maximum achieved throughput for all networks (↑)

Fig. 4: Comparison of inference execution times and maximum achieved throughput.

After thorough experimentation/validation on the youtube driver incidents dataset (YDID) that the baseline also used to test its performance (refer Table II), it was found that, TensorRT inference engine that was built on FP16 (half-precision floating-point) provided the best balance between inference speed and accuracy metrics for our particular use case over INT8. This conclusion is in line with the optimized FP16 operations of the Jetson AGX Orin's architecture. Fig. 4 shows the comparison of the mean inference execution time

and maximum achieved throughput between Schoonbeek *et al.* and our optimized approach.

After optimizing for the Jetson platform, our new RiskNET - Realtime pipeline was able to predict at 53 fps, which is significantly greater than the required minimum of 10 fps. In contrast, the original RiskNET model introduced by Schoonbeek *et al.* [5] was predicting at only 7 fps when run on the Jetson. This substantial improvement in frame rate demonstrates the effectiveness of our optimization efforts and the capability of our model to perform real-time risk assessment in practical scenarios.

E. Quantifying performance improvement post optimization

GPU memory operations play a critical role in determining the overall performance of a CUDA-based application, particularly for real-time systems that demand low-latency execution. The Nsight Systems (nsys) tool was used to validate that the model is ready to be deployed in realtime by profiling memory operations before and after optimization [35]. The model was optimized for real-time operations, where reducing memory transaction overhead is crucial to achieving low-latency performance. This analysis focuses on four key memory operations:

- **CUDA memset:** Initialization or resetting of memory on the GPU.
- **CUDA memcpy HtoD:** Host-to-Device memory transfer, which involves copying data from the CPU to the GPU.
- **CUDA memcpy DtoH:** Device-to-Host memory transfer, which copies data from the GPU back to the CPU.
- **CUDA memcpy DtoD:** Device-to-Device memory transfer, which copies data between different locations on the GPU.

Table I presents a comparative summary of these memory operations before and after optimization, based on metrics such as time spent, number of operations, and total memory usage.

The results indicate significant reductions in the number of memory operations across all categories after optimization. The CUDA memcpy HtoD operation saw a 78.9% reduction in count, while CUDA memset operations were reduced by 71.5%. This decrease in operation count, combined with larger average memory transfers, led to more efficient memory utilization and reduced overhead.

The optimization strategy incorporated several key improvements. Batch processing is evident in the HtoD operations, where fewer, larger transfers replaced multiple smaller ones. Memory reuse and improved management are reflected in the reduced CUDA memset operations, indicating more efficient initialization strategies. The dramatic 99.6% reduction in DtoD transfers suggests significantly improved data locality, crucial for enhancing cache utilization and reducing memory bandwidth consumption.

Although the time percentage for HtoD transfers increased from 48.2% to 71.1%, the overall time spent on key operations like CUDA memset decreased substantially. This trade-

TABLE I: CUDA GPU Memory Operations Summary (Before and After Optimization)

Operation	Metric	Before Opt.	After Opt.	Improvement
CUDA memset	Time (%)	37.5	26.9	-10.6%
	Count	7,027	2,003	-71.5%
	Total Mem (MB)	8,803.726	4,696.831	-46.7%
CUDA memcpy HtoD	Time (%)	48.2	71.1	+22.9%
	Count	4,736	1,000	-78.9%
	Total Mem (MB)	964.561	1,730.524	+79.4%
CUDA memcpy DtoH	Time (%)	4.5	1.9	-2.6%
	Count	3,997	998	-75.0%
	Total Mem (MB)	0.047	3.647	+7659.6%
CUDA memcpy DtoD	Time (%)	9.8	0.1	-9.7%
	Count	3,853	15	-99.6%
	Total Mem (MB)	133.003	0.589	-99.6%

TABLE II: Performance comparison of INT8 and FP16 precision TensorRT engines

TRT precision	Accuracy	AP	Recall	F1
INT8	0.532	0.850	0.873	0.825
FP16	0.725	0.885	0.893	0.888

off reflects a strategic reallocation of memory resources, prioritizing efficient bulk data movement from host to device. The reduction in operation counts, especially for CUDA memset and CUDA memcpy DtoH, contributed to enhanced performance by minimizing memory management overhead.

These optimizations are particularly crucial for real-time applications, where maintaining low-latency execution is essential. The reduction in operation counts and improvements in memory transfer efficiency enable better GPU resource utilization, which is critical for meeting real-time performance requirements. These enhancements in memory management are expected to contribute significantly to the overall responsiveness and efficiency of our collision risk prediction model in real-time scenarios, where rapid processing of incoming data is paramount.

IV. SIMULATION ENVIRONMENT

To validate our model in real time, the model has to be tested while running on the embedded platform. For a safe and rigorous validation, we validate our model optimized on the embedded device in a simulated closed-loop Hardware-In-the-Loop (HIL) setup. Since it is not possible to test AEBS on unsafe critical scenarios in the real-world environment, we leverage the simulation environment on Siemens' simcenter prescan [36]. The details of its implementation will be discussed in Section V.

For testing, we have created 4 new scenarios on the simcenter prescan, they consist of 1 near miss scenario, 2 scenarios with collisions, and 1 scenario with no collisions. These 4 scenarios are appropriately optimized to mimic diverse real-life situations. Since the spatio-temporal information adversely changes according to the scene settings and its dynamics in the real world, we have built these scenarios, each consisting of different road and weather conditions, and different scene semantics spreading across rural, sub-urban,



(a) Prescan Scenario 1

(b) Prescan Scenario 2



(c) Prescan Scenario 3

(d) Prescan Scenario 4

Fig. 5: The 4 prescan scenarios used for closed loop validation.

highway & urban scenarios. Most importantly, the velocity at which the ego vehicle, as well as the neighboring road users, are traveling, significantly impacts the performance of our model, thus testing that each of these scenarios has unique road users traveling at dis-similar velocities. These diverse scenarios are ultimately intended to test the robustness of our model across a range of aspects.

A. Prescan Scenario 1

Scenario 1 is set in a 4-lane highway, with bi-directional traffic road users (Fig.5a). The ego vehicle travels at a speed of 12.5m/s and follows a preceding SUV at a distance of 60m. The Preceding SUV comes to a complete halt after traveling for a short distance, upon which the ego vehicle collides with the SUV. There is continuous traffic on the opposite lanes.

B. Prescan Scenario 2

Scenario 2 is set in a thickly populated 2-lane urban setting, with bi-directional traffic intercepting a variety of

road intersections like roundabouts, left turns, right turns, merging etc (Fig.5b). The ego vehicle travels at a constant speed of 15m/s and travels along a predefined trajectory with no collisions. The ego vehicle intercepts a variety of traffic situations.

C. Prescan Scenario 3

Scenario 3 is set in a 2-lane sub-urban setting, with bi-directional traffic consisting of a variety of motor vehicles including a sports bike with a rider on it, a sedan and a hatchback on the opposite lane, a multi-utility vehicle (MUV) which is on the ego vehicle's lane (Fig.5c). The ego vehicle is traveling at a constant speed of 20m/s. A MUV merges onto the ego vehicle's lane from a service road, on which the ego vehicle collides.

D. Prescan Scenario 4

Scenario 4 is a 3-lane uni-directional road set in a rural setting (Fig.5d). The ego vehicle is sandwiched between a sports bike and a fire truck to the left and the right lane. A bicyclist crosses the road perpendicular to the ego vehicle and nearly misses the possibility of a collision. For an initial few moments, the bicyclist is occluded by the firetruck. The ego vehicle is traveling at a constant speed of 5.56m/s.

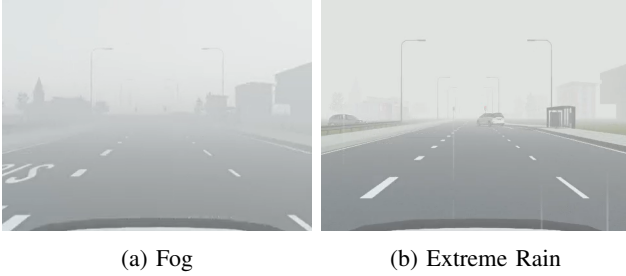


Fig. 6: Fog and extreme rain simulated on scenario 3.

To further create cosmetrical variation, we introduce the options of extreme rain and thick fog in all 4 scenarios. The visibility range during fog is set to 50 meters in prescan (Fig.6a). For rain, the rain dynamics is designed to match 384000 particles/cell with a raindrop diameter of 2.03mm (Fig.6b).

V. EXPERIMENTS

This section presents a comprehensive evaluation of our hardware specific, optimized collision risk prediction model. We begin with an examination of the kinematics of collision prevention. We then detail our Hardware-In-the-Loop (HIL) validation framework, which integrates our model with a sophisticated simulation environment. Finally, we provide a qualitative analysis of the model's performance across diverse scenarios under various environmental conditions. These experiments aim to thoroughly assess our model's capabilities, real-time performance characteristics, and potential for practical application in collision risk prediction systems.

A. Kinematics of Collision Prevention

When two vehicles are traveling at different speeds along the same path, various types of collisions can occur, such as head-on collisions, rear-end collisions, T-bone collisions, and others. For instance, in a rear-end collision, if vehicle A is traveling at a higher velocity (V_A) than vehicle B (V_B), the collision can be avoided by ensuring that the velocity of vehicle A is less than or equal to that of vehicle B. In other words, the relative velocity ΔV should be less than or equal to zero.

$$\Delta V = V_A - V_B \leq 0 \quad (2)$$

As proposed by the American Association of State Highway and Transportation Officials (AASHTO), the distance required by the ego vehicle to satisfy eq: 2 is termed as the Stopping Sight Distance (SSD) [15]. We chose to use AASHTO guidelines due to their comprehensive nature, regular updates, and wide recognition in the field of transportation engineering. These standards provide a robust, technically neutral approach that is particularly suitable for our work on automated systems and collision risk prediction, allowing for better comparability with international studies. SSD is mathematically represented as:

$$SSD = (0.278Vt) + 0.039 \frac{V^2}{a} \quad (3)$$

where V is the Velocity in km/h, t is the reaction time in seconds and a , the deceleration rate in m/s^2 . For our closed loop simulated validation, let's consider the reaction time to be 0. In that case, the braking distance simply becomes:

$$SSD = 0.039 \frac{V^2}{a} \quad (4)$$

In the context of our baseline model, RiskNET, it was trained on simulated datasets with custom labels. In any scenario, the spaces that each vehicle will occupy within the next 1.0 seconds are drawn for each frame. If there is an overlap in space, the situation is labeled unsafe. In a typical crossing where two vehicles pass near each other, this results in a transition from safe to unsafe to safe. For our example of a rear-end collision, the label would be 0 (safe) until the two vehicles are more than 1 second away. In this scenario, the model will predict a 1 (unsafe) only when the gap between them is less than 1 second. Thus, to avoid a potential collision, vehicle A has to decelerate to reduce the ΔV to ≤ 0 in ≤ 1 second.

Theoretically, eq.2 can only be satisfied for a limited range of V_A . As per laws of kinematics, the relation between the velocities, rate of acceleration, and time is defined as:

$$v = u + at \quad (5)$$

where v is V_A , u is V_B , a is the rate of deceleration of vehicle A and t is the time in seconds. This equation considers flat, dry roads with no sudden changes. The maximum possible

ΔV , such that vehicle A can avoid the collision in 1 second, can be found as:

$$\begin{aligned}\Delta V &= at \\ \Delta V_{max} &= a \cdot 1\end{aligned}\quad (7)$$



Fig. 7: Side view of the gap between the vehicles after coming to a complete stop. ($V_{ego} = 7.5\text{m/s}$)

Hence, the maximum ΔV possible in order to avoid a collision is simply equal to the rate of deceleration of vehicle A in m/s. The typical range of the rate of deceleration for a passenger vehicle is between $6\text{-}9\text{m/s}^2$. This translates to 21.6 km/h to 32.4 km/h . We tested this velocity range in our simulated closed-loop validation to check whether these conditions hold true.

The iterative experimentation process on Prescan scenario 1 revealed that the maximum achievable ΔV approached 7.5 m/s (27 km/h). HIL testing results indicated that any ΔV exceeding 27 km/h invariably resulted in a collision. Fig.7 illustrates the post-stop gap between the ego vehicle and the preceding vehicle. These results demonstrate that a hard braking deceleration rate of -9 m/s^2 was the minimum threshold required to bring the vehicle to a complete stop, thereby averting a collision.

The data clearly indicates that the vehicle was unable to prevent a collision at the initially estimated ΔV of 32.4 km/h . This limitation can be primarily attributed to the false negatives generated by the model, as evidenced in Scenario 1's results (4 lane highway setting) on Table III. As seen on Prescan Scenario 1's table, there are 6 false negatives out of the 10 true positives, this means that the model was not able to correctly identify risk for 6 of the total 10 frames spanning over 1 second. Thus, essentially losing out on 0.6 seconds of the total 1 second to react. These false negatives could've resulted in a delayed brake actuation, compromising the system's ability to prevent collisions at higher velocities.

It's important to acknowledge that not all collisions can be avoided, particularly at velocities exceeding the optimal ΔV . However, deceleration can significantly mitigate the impact on passengers. As per the European Commission, the risk of fatal injury for occupants of modern passenger cars (with crumple zone, airbags and seatbelts) is low when traveling at speeds below 50 km/h [37]. To examine this, we conducted a simulation of Prescan Scenario 3, involving a high-speed vehicle traveling at 20 m/s (72 km/h) colliding head-on with a merging vehicle. The system's intervention reduced

the impact velocity to 57 km/h , potentially improving the chances of survival.

This analysis underscores the critical relationship between model accuracy, particularly in terms of false negative rates, and the system's capacity to prevent collisions across a range of velocities. The findings highlight the importance of minimizing false negatives to enhance the overall safety performance of a collision avoidance system eg. Automated emergency braking system(AEBS).

B. Implementation Details

To rigorously evaluate the real-time performance of our collision risk prediction model, we implemented a HIL validation setup. This setup integrates an NVIDIA Jetson AGX Orin, which hosts our model, with a desktop computer running the Prescan simulation environment powered by MATLAB Simulink [38]. The two systems are interconnected via CAT 6 Ethernet cable, enabling high-speed, low-latency communication essential for real-time testing.

The Prescan simulation provides a realistic virtual environment for testing our model. A virtual camera sensor is mounted on the ego vehicle within the simulation, configured with a Field of View (FOV) of 46.21 degrees in azimuth and 35.48 degrees in elevation. This virtual camera generates RGB images that serve as input for our collision risk prediction model.

The data flow in our HIL setup is bidirectional, utilizing two different communication protocols. For transmitting image data from Simulink to the Jetson, we employ the Transmission Control Protocol (TCP). The camera block in Simulink generates RGB images, which are then serialized and transmitted over TCP. This protocol ensures reliable, ordered delivery of image data, crucial for maintaining the integrity of our input. Conversely, for sending prediction data from the Jetson back to Simulink, we use the User Datagram Protocol (UDP). The Jetson processes the received image through RiskNET, and the resulting prediction (a binary value: 0 for safe, 1 for unsafe) is sent back to Simulink over UDP. UDP is suitable for this small, time-sensitive data where occasional packet loss is acceptable.

The closed-loop (HIL) validation process operates as follows: The virtual camera in Prescan generates a new frame, which is serialized and transmitted to the Jetson via TCP. The Jetson deserializes the received data to reconstruct the image, which RiskNET then processes to generate a collision risk prediction. This prediction is sent back to the Simulink model via UDP. If the prediction is 1 (unsafe), Simulink actuates the brakes of the ego vehicle in the simulation. This process repeats in real-time for each new frame generated by the virtual camera, creating a closed-loop system where the model's predictions directly influence the simulation's behavior.

This HIL implementation offers several key advantages. It allows for testing of the model's performance under realistic timing constraints and accounts for hardware-specific performance characteristics by running the model on the target hardware. The closed-loop nature of the system, where

TABLE III: Comparison of closed loop real-time model performance metrics on different scenarios and conditions. 'No road user' represents condition with no road users other than the users involved in the critical scenario. 'TN', 'FP', 'FN', 'TP' stand for True Negative, False Positive, False Negative and True Positive respectively.

Scenario	No. of road users	Weather condition	AP	F1-score	Accuracy	Recall	TN	FP	FN	TP
Prescan Scenario 1 <i>4 lane highway setting</i>	1	Sunny	0.965	0.956	0.700	0.963	156	0	6	4
	6	Sunny	0.965	0.956	0.700	0.963	156	0	6	4
	6	Rain	0.965	0.956	0.700	0.963	156	0	6	4
	6	Fog	0.965	0.956	0.700	0.963	156	0	6	4
Prescan Scenario 2 <i>2 lane urban setting</i>	1	Sunny	0.976	0.964	0.978	0.959	200	9	0	12
	15	Sunny	0.966	0.927	0.952	0.909	189	20	0	12
	15	Rain	0.960	0.926	0.912	0.909	190	19	1	11
	15	Fog	0.968	0.937	0.959	0.923	192	17	0	12
Prescan Scenario 3 <i>2 lane sub-urban setting</i>	1	Sunny	0.957	0.946	0.700	0.955	126	0	6	4
	8	Sunny	0.919	0.922	0.684	0.926	122	4	6	4
	8	Rain	0.919	0.922	0.684	0.926	122	4	6	4
	8	Fog	0.919	0.922	0.684	0.926	122	4	6	4
Prescan Scenario 4 <i>3 lane rural setting</i>	1	Sunny	0.994	0.993	0.996	0.992	263	2	0	9
	3	Sunny	0.994	0.993	0.996	0.992	263	2	0	9
	3	Rain	0.989	0.986	0.992	0.985	261	4	0	9
	3	Fog	0.988	0.983	0.990	0.981	260	5	0	9

predictions can affect the simulation via brake actuation, allows for comprehensive testing of the model's impact on vehicle behavior. Furthermore, this setup enables safe and repeatable testing of dangerous scenarios without real-world risks.

VI. RESULTS

The performance of our collision risk prediction model was evaluated across four diverse scenarios in a Hardware-In-the-Loop (HIL) simulation environment using Siemens' Simcenter Prescan (Table III). Each scenario was tested under various conditions: with no extraneous road users, with all road users, and with all road users under adverse weather conditions (rain and fog). This comprehensive testing approach allowed us to assess the model's robustness and adaptability to different environmental factors.

1) *Prescan Scenario 1 (Highway Setting)*: The model exhibited consistent performance across all conditions, with an AP of 0.965 and an F1-score of 0.956. This consistency can be attributed to the relatively uncomplicated nature of highway environments, where vehicle movements are more predictable and the scene is less cluttered. The model's resilience to weather conditions in this scenario suggests that the reduced visibility from rain or fog did not significantly impact its ability to assess collision risks in a highway setting. However, the lower accuracy (0.700) indicates potential challenges in distinguishing between critical and non-critical situations, possibly due to the high-speed nature of highway driving.

2) *Prescan Scenario 2 (Urban Setting)*: In this complex urban environment, the model demonstrated its highest performance with no additional road users. The introduction of additional road users and adverse weather conditions led to a slight performance decrease (AP range: 0.960-

0.968, F1-score range: 0.926-0.937). This decline can be attributed to the increased complexity of urban scenarios, where numerous road users, intersections, and varying traffic patterns create a more challenging prediction environment. The more pronounced impact of rain compared to fog (lowest AP and F1-score in rainy conditions) suggests that the model may be more sensitive to changes in visual patterns caused by rain than to reduced visibility alone.

3) *Prescan Scenario 3 (Suburban Setting)*: The model's performance in this scenario remained consistent across different conditions but showed a decrease compared to other scenarios (AP dropped from 0.957 to 0.919). This decline, along with the lower accuracy (0.684), can be attributed to the unique challenges presented by suburban environments, such as varying road widths, mixed vehicle types, and unexpected merging situations. The consistent performance across weather conditions suggests that the model's limitations in this scenario are more related to the environment's inherent complexity than to weather-induced visibility issues.

4) *Prescan Scenario 4 (Rural Setting)*: The model demonstrated its strongest and most consistent performance in this scenario. This robust performance can be attributed to the relative simplicity of rural settings, with fewer road users and less complex traffic patterns. The model's ability to maintain high performance even with a partially occluded bicyclist suggests effective handling of sudden appearances in less cluttered environments. The minor performance drops in adverse weather conditions indicate that while the model is generally robust, there is still a slight impact from reduced visibility or changed visual cues in rural settings.

5) *False Positives and Negatives Analysis*: The model maintained a consistently low false negative rate across all scenarios, which is crucial for safety-critical applications. However, the increase in false positives in Scenario

2 with additional road users and adverse weather indicates a potential over-sensitivity in complex urban environments. This behavior suggests that the model is cautious in high-complexity situations, prioritizing safety at the cost of some false alarms.

VII. DISCUSSION

This study aimed to enhance the real-time capabilities of collision risk prediction. Our findings demonstrate that low-latency, real-time collision prediction is achievable through careful optimization of both software and hardware components. The optimization of CUDA memory operations, leveraging the tensor and CUDA cores along with NVIDIA's TensorRT, significantly reduced inference time and increased throughput. While optimizing the model for edge devices affected its performance metrics, the trade-off between performance and real-time capability was generally favorable, with the model maintaining robust performance across various weather conditions. In the context of Autonomous Emergency Braking Systems (AEBS), our research indicates that while significant improvements in collision avoidance are possible, achieving 100% prevention of all accidents remains challenging. This is partly due to the core limitation of the RiskNET model, which can only predict for the next one second. This short prediction window is insufficient for anticipating accidents that may develop over a longer time frame. Future work should focus on extending this prediction window to enable earlier detection of potential risks. A potential improvement could involve introducing an intermediate class between 0 (safe) and 1 (unsafe), representing the possibility of unsafe situations developing in the future. This would allow the model to flag potential risks earlier, providing more time for preventive actions. Future research directions should include enhancing model sensitivity to reduce false negatives, exploring more advanced hardware solutions and optimization techniques, investigating the integration of additional sensor modalities, and developing more sophisticated simulation environments for robust testing and validation. Additionally, research should focus on extending the prediction window and incorporating multi-class risk assessment to improve the model's ability to anticipate and prevent accidents over longer time horizons.

VIII. CONCLUSION

This research shows real-time collision risk prediction through optimization, particularly by leveraging TensorRT for model acceleration. However, the current training methodology of the RiskNET model presents significant limitations for deployment in complex scenarios. The original model was trained on simplified datasets with constant velocities and radius assumptions, which do not adequately reflect real-world complexities. Additionally, the validation process relied on the small YDID dataset, containing only 13 positive samples, raising concerns about the model's generalizability.

While our hardware-in-the-loop validation provided insights into the model's real-time capabilities, it also high-

lighted that the model is not sufficiently trained for diverse traffic situations. Nevertheless, the optimization methods employed lay a strong foundation for future work. As richer training datasets become available, they will be crucial for enhancing the model's robustness. Furthermore, expanding the scope to include additional input modalities from various sensor types will likely improve performance and adaptability.

IX. ACKNOWLEDGMENT

I acknowledge the use of Grammarly in helping me review and improve the writing in this report. I used the following features, Clarity, Correctness, Grammar and spelling checks. I reviewed the feedback generated by Grammarly critically and, based on this, revised the writing using my own words and expressions.

REFERENCES

- [1] A. Gujarathi, A. Kulkarni, U. Patil, Y. Phalak, R. Deotalu, A. Jain, N. Panchi, A. Dhabale, and S. Chiddarwar, "Design and Development of Autonomous Delivery Robot," 2021, version Number: 1. [Online]. Available: <https://arxiv.org/abs/2103.09229>
- [2] C. Jb, "Effectiveness of forward collision warning and autonomous emergency braking systems in reducing front-to-rear crash rates," *Accident; analysis and prevention*, vol. 99, no. Pt A, Feb. 2017, publisher: Accid Anal Prev. [Online]. Available: <https://pubmed.ncbi.nlm.nih.gov/27898367/>
- [3] E. NCAP, "European new car assessment programme: Test protocol - aeb car-to-car systems," European New Car Assessment Programme, Technical Report Version 4.0.1, 2022.
- [4] M. Distner, M. Bengtsson, T. Broberg, and L. Jakobsson, "City safety—a system addressing rear-end collisions at low speeds," *ESV Conference*, pp. 09–0371, 2009.
- [5] T. J. Schoonbeek, F. J. Piva, H. R. Abdolhay, and G. Dubbelman, "Learning to Predict Collision Risk from Simulated Video Data," in *2022 IEEE Intelligent Vehicles Symposium (IV)*. Aachen, Germany: IEEE, Jun. 2022, pp. 943–951. [Online]. Available: <https://ieeexplore.ieee.org/document/9827228/>
- [6] T. Schoonbeek, "Learning to Predict Collision Risk from Simulated Optical Flow," May 2021. [Online]. Available: <https://research.tue.nl/en/studentTheses/learning-to-predict-collision-risk-from-simulated-optical-flow>
- [7] A. Eidehall and J. Pohl, "Method and system for collision avoidance," US Patent US8 112 225B2, Feb., 2012. [Online]. Available: <https://patents.google.com/patent/US8112225B2/en>
- [8] G. Stein, E. Dagan, O. Mano, and A. Shashua, "Collision warning system," US Patent US9 656 607B2, May, 2017. [Online]. Available: <https://patents.google.com/patent/US9656607B2/en>
- [9] X. Yang and X. Liu, "Design of a wide-beam microstrip array antenna for automotive radar application," *IEEE Access*, vol. PP, pp. 1–1, 10 2021.
- [10] F.-H. Chan, Y.-T. Chen, Y. Xiang, and M. Sun, "Anticipating accidents in dashcam videos," in *Asian Conference on Computer Vision*. Springer, 2016, pp. 136–153.
- [11] Z. Chen, Y. Wang, W. Han, R. Feng, and J. Chen, "An improved pretraining strategy-based scene classification with deep learning," *IEEE Geoscience and Remote Sensing Letters*, vol. 17, no. 5, pp. 844–848, 2020.
- [12] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. Zitnick, "Microsoft coco: Common objects in context," 05 2014.
- [13] Meta AI, "Papers with Code - ImageNet Benchmark (Image Classification)." [Online]. Available: <https://paperswithcode.com/sota/image-classification-on-imagenet>
- [14] Y. Zhou and K. Yang, "Exploring TensorRT to Improve Real-Time Inference for Deep Learning," in *2022 IEEE 24th Int Conf on High Performance Computing & Communications; 8th Int Conf on Data Science & Systems; 20th Int Conf on Smart City; 8th Int Conf on Dependability in Sensor, Cloud & Big Data Systems & Application (HPCC/DSS/SmartCity/DependSys)*. Hainan,

- China: IEEE, Dec. 2022, pp. 2011–2018. [Online]. Available: <https://ieeexplore.ieee.org/document/10074837/>
- [15] A. A. o. S. Highway and T. Officials, *A Policy on Geometric Design of Highways and Streets, 1984*. American Association of State Highway and Transportation Officials, 1984. [Online]. Available: <https://books.google.nl/books?id=ISISAAAAMAAJ>
 - [16] T. P. Swaminathan, C. Silver, and T. Akilan, “Benchmarking Deep Learning Models on NVIDIA Jetson Nano for Real-Time Systems: An Empirical Investigation,” Jun. 2024. [Online]. Available: <https://arxiv.org/abs/2406.17749v1>
 - [17] M. H. Zhu and S. Gupta, “To prune, or not to prune: Exploring the efficacy of pruning for model compression,” 2018. [Online]. Available: <https://openreview.net/forum?id=S1IN69AT->
 - [18] A. Polino, R. Pascanu, and D. Alistarh, “Model compression via distillation and quantization,” in *International Conference on Learning Representations*, 2018. [Online]. Available: <https://openreview.net/forum?id=S1XoIQbRW>
 - [19] U. Gupta, Y. Kim, S. Lee, J. Tse, H. Lee, G. Wei, D. Brooks, and C. Wu, “Chasing carbon: The elusive environmental footprint of computing,” in *Proceeding - 27th IEEE International Symposium on High Performance Computer Architecture, HPCA 2021*, ser. Proceedings - International Symposium on High-Performance Computer Architecture. IEEE Computer Society, Feb. 2021, pp. 854–867, publisher Copyright: © 2021 IEEE.; 27th Annual IEEE International Symposium on High Performance Computer Architecture, HPCA 2021 ; Conference date: 27-02-2021 Through 01-03-2021.
 - [20] K. Hara, H. Kataoka, and Y. Satoh, “Can Spatiotemporal 3D CNNs Retrace the History of 2D CNNs and ImageNet?” 2017, version Number: 2. [Online]. Available: <https://arxiv.org/abs/1711.09577>
 - [21] S. Xie, R. Girshick, P. Dollár, Z. Tu, and K. He, “Aggregated Residual Transformations for Deep Neural Networks,” 2016, version Number: 2. [Online]. Available: <https://arxiv.org/abs/1611.05431>
 - [22] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” 2015, version Number: 1. [Online]. Available: <https://arxiv.org/abs/1512.03385>
 - [23] J. Zhang, Z. Chen, G. Yan, Y. Wang, and B. Hu, “Faster and Lightweight: An Improved YOLOv5 Object Detector for Remote Sensing Images,” *Remote Sensing*, vol. 15, no. 20, p. 4974, Oct. 2023. [Online]. Available: <https://www.mdpi.com/2072-4292/15/20/4974>
 - [24] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks,” Jun. 2015. [Online]. Available: <https://arxiv.org/abs/1506.01497v3>
 - [25] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” 06 2016, pp. 779–788.
 - [26] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, “YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors,” Jul. 2022. [Online]. Available: <https://arxiv.org/abs/2207.02696v1>
 - [27] A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, and G. Ding, “YOLOv10: Real-Time End-to-End Object Detection,” May 2024, arXiv:2405.14458 [cs] version: 1. [Online]. Available: <http://arxiv.org/abs/2405.14458>
 - [28] “Jetson - Embedded AI Computing Platform.” [Online]. Available: <https://developer.nvidia.com/embedded-computing>
 - [29] Google, “Google Cloud, Cloud tensor processing units (TPUs).” [Online]. Available: <https://cloud.google.com/tpu/docs/intro-to-tpu>
 - [30] NVIDIA Corporation, “NVIDIA TensorRT.” [Online]. Available: <https://developer.nvidia.com/tensorrt>
 - [31] Meta AI, “PyTorch - From research to production.” [Online]. Available: <https://pytorch.org/>
 - [32] D.-J. Shin and J.-J. Kim, “A Deep Learning Framework Performance Evaluation to Use YOLO in Nvidia Jetson Platform,” *Applied Sciences*, vol. 12, no. 8, 2022. [Online]. Available: <https://www.mdpi.com/2076-3417/12/8/3734>
 - [33] NVIDIA Corporation, “Nvidia tensorrt developer guide — nvidia docs.” [Online]. Available: <https://docs.nvidia.com/deeplearning/tensorrt/pdf/TensorRT-Developer-Guide.pdf>
 - [34] Leela S. Karumbunathan, “NVIDIA Jetson AGX Orin Series.” [Online]. Available: <https://www.nvidia.com/content/dam/en-zz/Solutions/gtc21/jetson-orin/nvidia-jetson-agx-orin-technical-brief.pdf>
 - [35] Nvidia nsight systems. [Online]. Available: <https://developer.nvidia.com/nsight-systems>
 - [36] Siemens simcenter prescan. [Online]. Available: <https://plm.sw.siemens.com/en-US/simcenter/autonomous-vehicle-solutions/prescan/>
 - [37] European Road Safety Observatory, “Road safety thematic report

- speeding,” European Commission, Brussels, Thematic Report, 2021. [Online]. Available: https://road-safety.transport.ec.europa.eu/document/download/5a6206ab-2624-41f4-8de9-180e7335911d_en

[38] (2023) Matlab® simulink. [Online]. Available: <https://mathworks.com/products/simulink.html>